BABU BANARASI DAS UNIVERSITY

2026-2027



SCHOOL OF COMPUTER APPLICATIONS

No SQL Project

Submitted to:
Mr. Harendra Singh
Department: BCA (DS & AI)
Date: 25th September

Submitted by:
MOHD TAUSEEF
Roll no: (32)
university roll no:
1250258274

INDEX

S.No.	OPERATION	DATE
1	CREATE DATABASE	18 Aug 2026
2	CREATE COLLECTION	19 Aug 2026
3	DISPLAY ALL STUDENTS	25 Aug 2026
4	EQUAL TO — $eq	25 Aug 2026
5	GREATER THAN — $gt	01 Sep 2026
6	LESS THAN — $lt	02 Sep 2026
7	GREATER THAN OR EQUAL TO — $gte	08 Sep 2026
8	LESS THAN OR EQUAL TO — $lte	09 Sep 2026
9	UPDATE ONE OPERATION	15 Sep 2026
10	DELETE ONE OPERATION	15 Sep 2026
11	AND OPERATION — $and	16 Sep 2026
12	OR OPERATION — $or	16 Sep 2026

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled “Student Management System Using MongoDB”.

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, Sorting and delete data. I also learned how different query operators such as greater than, less than, equal to, AND, and OR are used in MongoDB.

I am thankful to my teacher for the guidance and support provided throughout the completion of this project.

INTRODUCTION

MongoDB is a popular NoSQL database management system that stores data in the form of documents. Unlike traditional relational databases, MongoDB does not require data to be stored in fixed rows and columns.

In this project, a Student Management System has been created using MongoDB. The database contains information about 50 students, including their roll number, name, age, marks, and city.

Different MongoDB operations have been performed on the student data, such as inserting, displaying, updating, deleting, sorting, counting, and searching records. Comparison operators such as $eq, $gt, $lt, $gte, and $lte have also been demonstrated.

OBJECTIVES

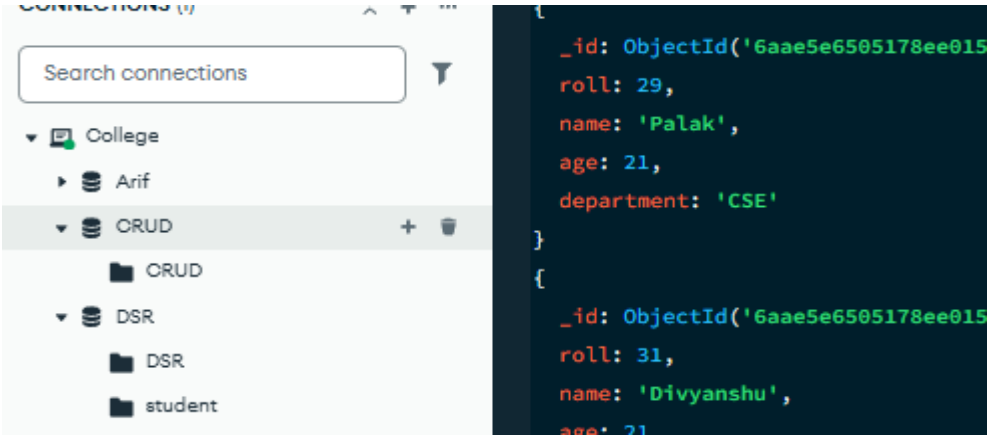
- To understand the concept of NoSQL databases.
- To learn the basic features of MongoDB.
- To create a database and collection.
- To insert 40 student records.
- To perform CRUD operations.
- To use comparison operators.
- To perform sorting and filtering.
- To understand AND and OR operations.

1. CREATE DATABASE

MongoDB Query:

use CRUD

Explanation: Creates or switches to the CRUD database.

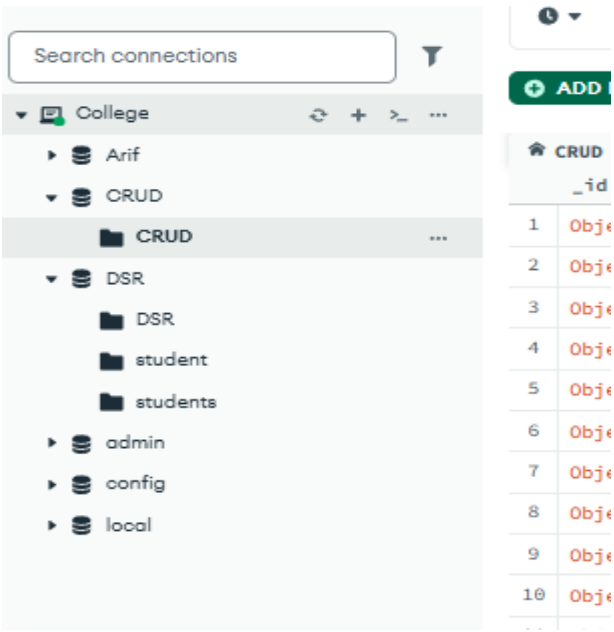


2. CREATE COLLECTION

MongoDB Query:

db. create Collection("CRUD")

Explanation: Creates a collection named CRUD.



3. DISPLAY ALL STUDENTS

MongoDB Query:

db. CRUD. Find ().

```
}
> db.CRUD.find()
< {
    _id: ObjectId('6aae544e85178ee815c9caf2'),
    rollno: 1,
    name: 'Sam',
    age: 20,
    department: 'ECA'
  }
  {
    _id: ObjectId('6aae5e6585178ee815c9caf3'),
    rollno: 2,
    name: 'Ram',
    age: 20,
    department: 'ECA'
  }
  {
    _id: ObjectId('6aae5e6585178ee815c9caf4'),
    roll: 3,
    name: 'Adi',
    age: 20,
    department: 'ECA'
  }
```

Explanation: Retrieves documents from the collection

4. EQUAL TO — $eq

MongoDB Query:

db. students. Find ({marks: {$eq: 78}})



Explanation: Finds students whose marks are exactly 78.

5. GREATER THAN — $gt

MongoDB Query:

db.students.find({marks: {$gt: 85}})

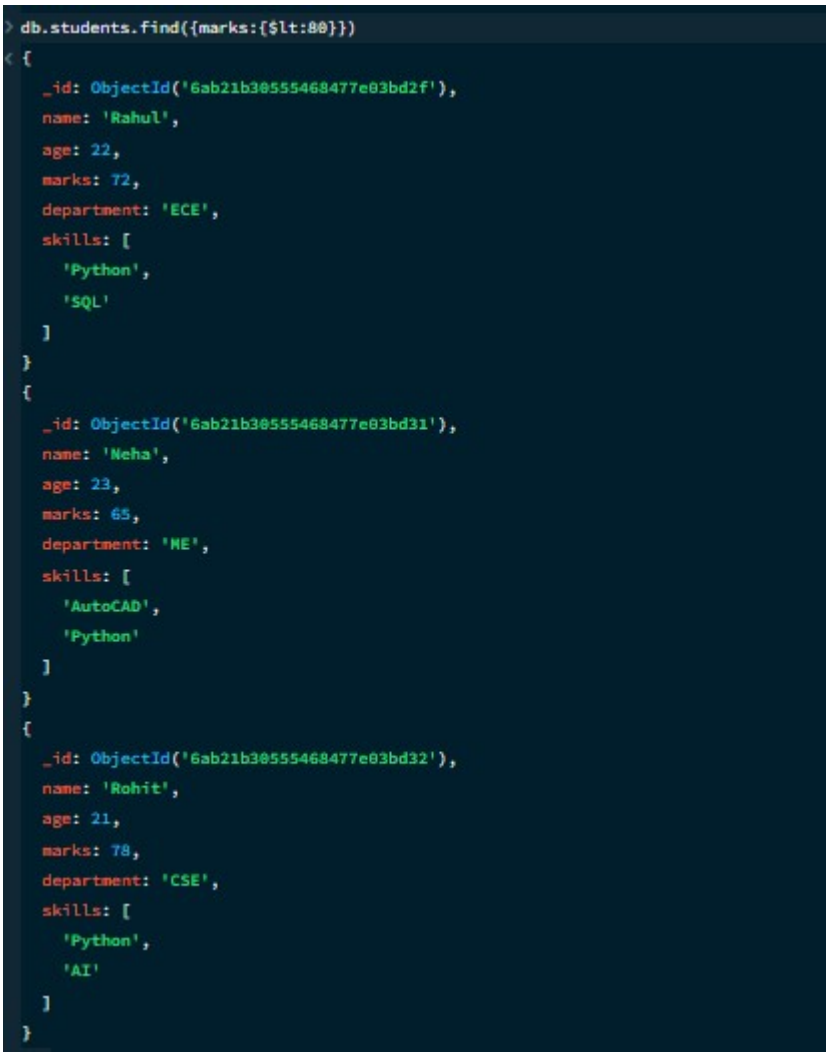
```
> db.students.find({marks: {$gt: 85}})
< {
    _id: ObjectId('6aae9ea78f0f98eb52df221c'),
    roll: 1,
    name: 'Anamika',
    age: 19,
    marks: 88,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df221e'),
    roll: 3,
    name: 'Vaishnavi',
    age: 19,
    marks: 92,
    city: 'Kanpur'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2223'),
    roll: 8,
    name: 'Pratik',
    age: 19,
    marks: 90,
    city: 'Lucknow'
  }
  {
    _id: ObjectId('6aae9ea78f0f98eb52df2227'),
    roll: 12,
    name: 'Shreya',
    age: 19,
    marks: 95,
    city: 'Lucknow'
  }
```

Explanation: Finds students whose marks are greater than 85.

6. LESS THAN — $lt

MongoDB Query:

db. students. find ({marks: {$lt: 80}})

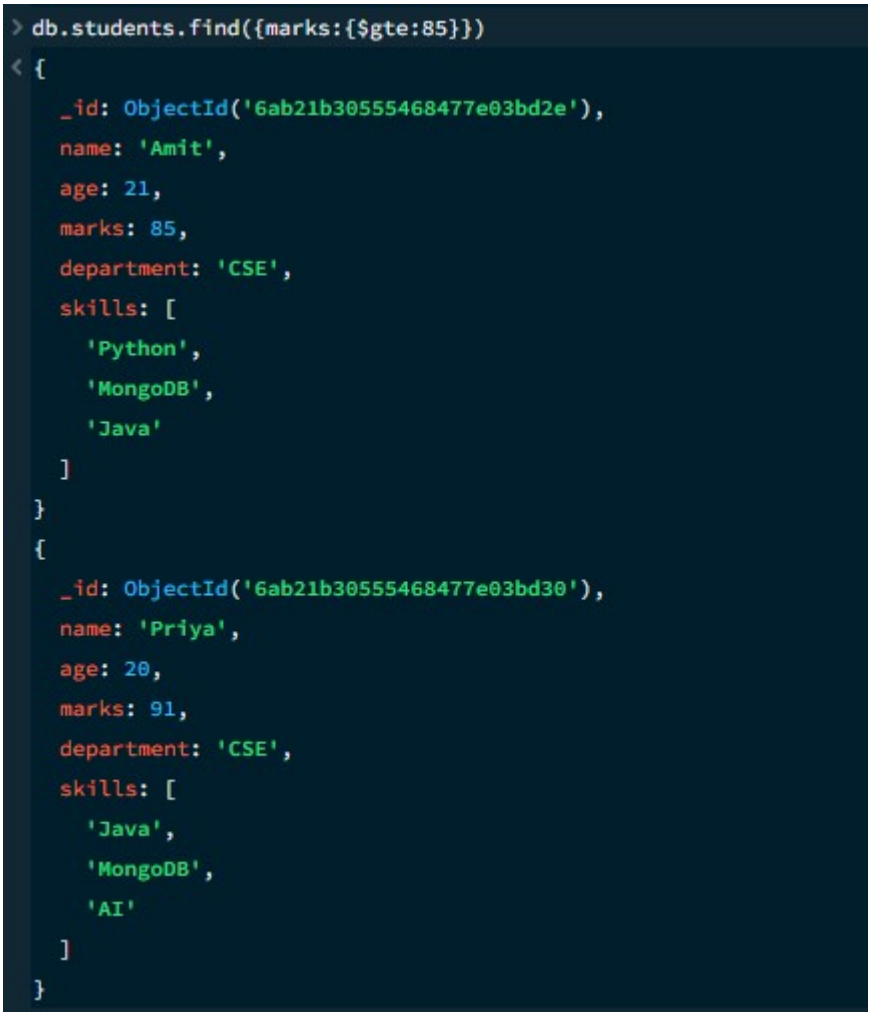


Explanation: Finds students whose marks are less than 80.

7. GREATER THAN OR EQUAL TO — $gte

MongoDB Query:

db. students. find({marks: {$gte: 85}})



Explanation: Finds students whose marks are 90 or above.

8. LESS THAN OR EQUAL TO — $lte

MongoDB Query:

db. students. find({marks: {$lte: 85}})

```
> db.students.find({marks:{$lte:85}})
< {
    _id: ObjectId('6ab21b30555468477e03bd2e'),
    name: 'Amit',
    age: 21,
    marks: 85,
    department: 'CSE',
    skills: [
      'Python',
      'MongoDB',
      'Java'
    ]
  }
  {
    _id: ObjectId('6ab21b30555468477e03bd2f'),
    name: 'Rahul',
    age: 22,
    marks: 72,
    department: 'ECE',
    skills: [
      'Python',
      'SQL'
    ]
  }
```

Explanation: Finds students whose marks are 85 or below.

9. UPDATE ONE OPERATION

MongoDB Query:

```
db.CRUD.updateOne(
  {roll:31},
  {$set:{department:"ECA"}}
)
```

```
db.CRUD.updateOne({roll:31},{$set:{department:"ECA"}})
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

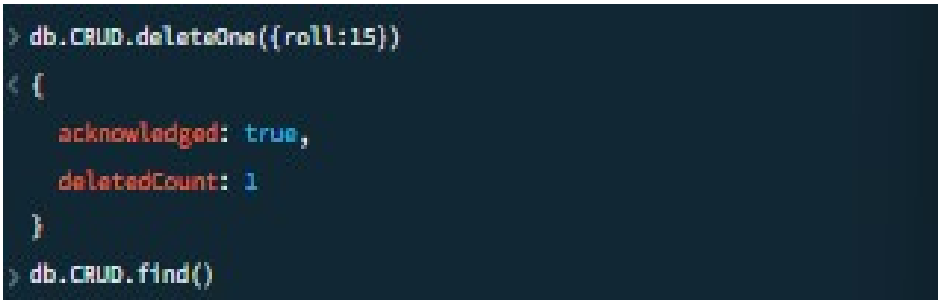
```
{
  _id: ObjectId('6aae5e6585178ec015c9cb10'),
  roll: 31,
  name: 'Divyanshu',
  age: 21,
  department: 'ECA'
}
```

Explanation: Updates the department to ECA of the student with roll number 31.

10. DELETE ONE OPERATION

MongoDB Query:

db.CRUD.deleteOne({roll:15})

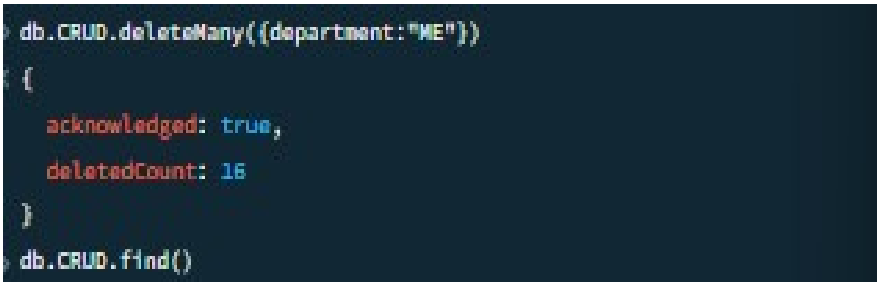


Explanation: Deletes one document matching roll number 10.

12. DELETE MANY OPERATION

MongoDB Query:

db. CRUD. deleteMany({
  department:”ME”
})



Explanation: Deletes all students having marks below 60.

11. AND OPERATION — $and

MongoDB Query:

```
db. students. find({
  $and:[
    {marks:{$gt:70}},
    {age:23}
  ]
})
```

```
> db.students.find({$and:[{marks:{$gt:70}},{age:{$lt:23}}]})
< {
    _id: ObjectId('6ab21b30555468477e03bd2e'),
    name: 'Amit',
    age: 21,
    marks: 85,
    department: 'CSE',
    skills: [
      'Python',
      'MongoDB',
      'Java'
    ]
  }
  {
    _id: ObjectId('6ab21b30555468477e03bd2f'),
    name: 'Rahul',
    age: 22,
    marks: 72,
    department: 'ECE',
    skills: [
      'Python',
      'SQL'
    ]
  }
```

Explanation: Finds students satisfying both conditions.

12. OR OPERATION — $or

MongoDB Query:

```
db.students.find({
  $or:[
    {department:"CSE"},
    { department:"ECA "}
  ]
})
```

```
db.students.find({$or:[{department:"CSE"},{department:"ECA"}]})
{
  _id: ObjectId('6ab21b30555468477e03bd2e'),
  name: 'Amit',
  age: 21,
  marks: 85,
  department: 'CSE',
  skills: [
    'Python',
    'MongoDB',
    'Java'
  ]
}
{
  _id: ObjectId('6ab21b30555468477e03bd30'),
  name: 'Priya',
  age: 20,
  marks: 91,
  department: 'CSE',
  skills: [
    'Java',
    'MongoDB',
    'AI'
  ]
}
```

Explanation: Finds students from CSE or ECA.

MONGODB OPERATORS USED:

Operator	Meaning	Example
insertOne **insertMany**	Create one and many	insertOne({name:"Arif"}) insertMany([{name:"Adeel"},{name:"Arif"}])
Find()	Read	Find()
updateOne **updateMany**	Update one and many	updateOne({},{{}}) updateMany({},{{}})
deleteOne **deleteMany**	Delete one and many	Delete One({}) Delete Many({})
$eq	Equal to	marks: {$eq:88}
$gt	Greater than	marks: {$gt:85}
$lt	Less than	marks: {$lt:60}
$gte	Greater than or equal to	marks: {$gte:90}
$lte	Less than or equal to	marks: {$lte:70}
$and	All conditions true	$and:[...]

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Student Management System containing student information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how to Ascending order & descending order data, comparison operators such as $eq, $gt, $lt, $gte, and $lte work, along with logical operators such as $and and $or.

Overall, this project helped me understand how MongoDB can be used to manage and query student data efficiently.